```
"""
This cell installs the required NLP and visualization libraries.
- nltk: for tokenization and POS tagging
- spacy: for advanced NLP and universal POS tags
- matplotlib: for data visualization
"""

!pip install nltk spacy matplotlib
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.21.1)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (25.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.1)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1,
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spac
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->spacy) (3.0.3)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8.0.0,>=5.2.1->weasel<0.5.0
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 55.1 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

Task

Replace the content of code cell `qLvC2YpLHogN` with:

```
import nltk # Import the Natural Language Toolkit (NLTK) library for text processing tasks.
import spacy # Import spaCy for advanced natural language processing functionalities.
import matplotlib.pyplot as plt # Import matplotlib.pyplot for creating visualizations, such as bar charts.
from collections import Counter # Import Counter from collections to easily count the frequency of items in a list.
```

```
nltk.download('punkt') # Download the 'punkt' tokenizer model, essential for splitting text into sentences and words.
nltk.download('averaged_perceptron_tagger') # Download the 'averaged_perceptron_tagger' model, used for part-of-speech tagg
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
True
```

```
essay_text = """
Academic writing is essential in higher education.
It enables students to develop critical thinking and research skills.
Scholars present arguments logically and support them with evidence.
Effective academic writing improves communication and knowledge sharing.
""" # Define a multi-line string variable `essay_text` containing the academic essay content for analysis.
```

```
print(essay_text) # Print the raw academic essay text to display its content.
```

```
Academic writing is essential in higher education.
It enables students to develop critical thinking and research skills.
Scholars present arguments logically and support them with evidence.
Effective academic writing improves communication and knowledge sharing.
```

```
nltk.download('averaged_perceptron_tagger_eng') # Download an English-specific averaged perceptron tagger for improved POS
nltk_pos_tags = nltk.pos_tag(tokens) # Perform Part-of-Speech (POS) tagging on the tokenized essay text using NLTK.
nltk_pos_tags # Display the list of tokens with their corresponding NLTK POS tags.
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[('Academic', 'NNP'),
 ('writing', 'NN'),
 ('is', 'VBZ'),
 ('essential', 'JJ'),
 ('in', 'IN'),
 ('higher', 'JJR'),
 ('education', 'NN'),
 ('.', '.'),
 ('It', 'PRP'),
 ('enables', 'VBZ'),
 ('students', 'NNS'),
 ('to', 'TO'),
 ('develop', 'VB'),
 ('critical', 'JJ'),
 ('thinking', 'NN'),
 ('and', 'CC'),
 ('research', 'NN'),
 ('skills', 'NNS'),
 ('.', '.'),
 ('Scholars', 'NNP'),
 ('present', 'JJ'),
 ('arguments', 'NNS'),
 ('logically', 'RB'),
 ('and', 'CC'),
 ('support', 'VB'),
 ('them', 'PRP'),
 ('with', 'IN'),
 ('evidence', 'NN'),
 ('.', '.'),
 ('Effective', 'JJ'),
 ('academic', 'JJ'),
 ('writing', 'VBG'),
 ('improves', 'NNS'),
 ('communication', 'NN'),
 ('and', 'CC'),
 ('knowledge', 'NN'),
 ('sharing', 'NN'),
 ('.', '.')]
```

```
nlp = spacy.load("en_core_web_sm") # Load the English language model from spaCy for advanced NLP tasks.
doc = nlp(essay_text) # Process the essay text with the loaded spaCy model to create a Doc object.

spacy_pos_tags = [(token.text, token.pos_) for token in doc] # Extract tokens and their Universal Part-of-Speech (POS) tags
spacy_pos_tags # Display the list of tokens with their corresponding spaCy POS tags.
```

```
[('\n', 'SPACE'),
 ('Academic', 'ADJ'),
 ('writing', 'NOUN'),
 ('is', 'AUX'),
 ('essential', 'ADJ'),
 ('in', 'ADP'),
 ('higher', 'ADJ'),
 ('education', 'NOUN'),
```

```
 ('.', 'PUNCT'),
 ('\n', 'SPACE'),
 ('It', 'PRON'),
 ('enables', 'VERB'),
 ('students', 'NOUN'),
 ('to', 'PART'),
 ('develop', 'VERB'),
 ('critical', 'ADJ'),
 ('thinking', 'NOUN'),
 ('and', 'CCONJ'),
 ('research', 'NOUN'),
 ('skills', 'NOUN'),
 ('.', 'PUNCT'),
 ('\n', 'SPACE'),
 ('Scholars', 'NOUN'),
 ('present', 'ADJ'),
 ('arguments', 'NOUN'),
 ('logically', 'ADV'),
 ('and', 'CCONJ'),
 ('support', 'VERB'),
 ('them', 'PRON'),
 ('with', 'ADP'),
 ('evidence', 'NOUN'),
 ('.', 'PUNCT'),
 ('\n', 'SPACE'),
 ('Effective', 'ADJ'),
 ('academic', 'ADJ'),
 ('writing', 'NOUN'),
 ('improves', 'VERB'),
 ('communication', 'NOUN'),
 ('and', 'CCONJ'),
 ('knowledge', 'NOUN'),
 ('sharing', 'NOUN'),
 ('.', 'PUNCT'),
 ('\n', 'SPACE')]
```

```
nouns = [token.text.lower() for token in doc if token.pos_ == "NOUN"] # Extract all tokens identified as nouns by spaCy and
verbs = [token.text.lower() for token in doc if token.pos_ == "VERB"] # Extract all tokens identified as verbs by spaCy and

nouns, verbs # Display the extracted lists of nouns and verbs.
```

```
(['writing',
  'education',
  'students',
  'thinking',
  'research',
  'skills',
  'scholars',
  'arguments',
  'evidence',
  'writing',
  'communication',
  'knowledge',
  'sharing'],
 ['enables', 'develop', 'support', 'improves'])
```

```
noun_freq = Counter(nouns) # Calculate the frequency of each noun using Counter.
verb_freq = Counter(verbs) # Calculate the frequency of each verb using Counter.

noun_freq, verb_freq # Display the frequency counts for nouns and verbs.
```

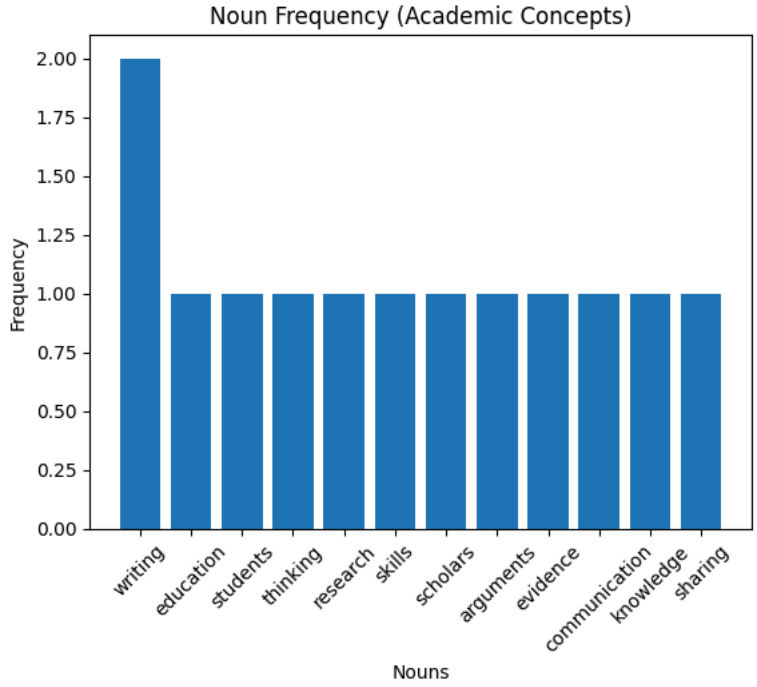
```
(Counter({'writing': 2,
          'education': 1,
          'students': 1,
          'thinking': 1,
          'research': 1,
          'skills': 1,
          'scholars': 1,
          'arguments': 1,
          'evidence': 1,
          'communication': 1,
          'knowledge': 1,
          'sharing': 1}),
 Counter({'enables': 1, 'develop': 1, 'support': 1, 'improves': 1}))
```

```
print("Noun Frequencies:") # Print a header for noun frequencies.
for noun, freq in noun_freq.items(): # Iterate through the noun_freq dictionary.
    print(noun, ":", freq) # Print each noun and its corresponding frequency.

print("\nVerb Frequencies:") # Print a header for verb frequencies, with a newline for separation.
for verb, freq in verb_freq.items(): # Iterate through the verb_freq dictionary.
    print(verb, ":", freq) # Print each verb and its corresponding frequency.
```

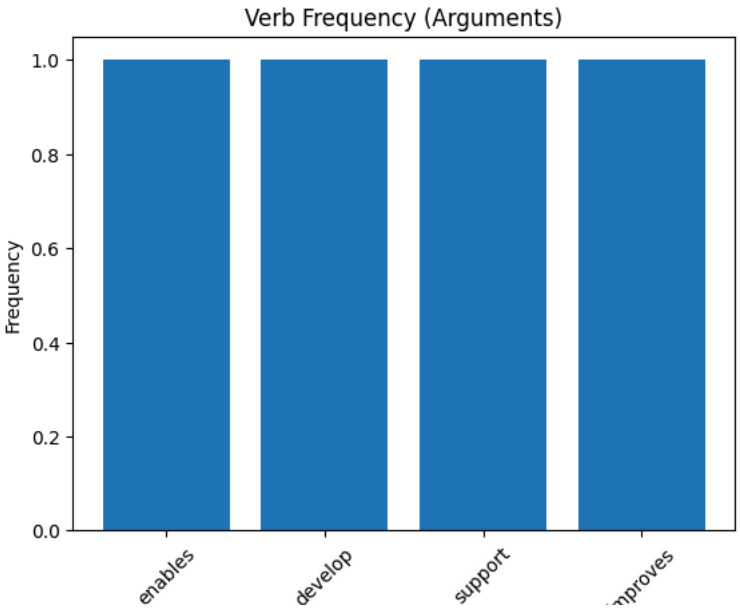
```
Noun Frequencies:
writing : 2
education : 1
```

```
students : 1
thinking : 1
research : 1
skills : 1
scholars : 1
arguments : 1
evidence : 1
communication : 1
knowledge : 1
sharing : 1

Verb Frequencies:
enables : 1
develop : 1
support : 1
improves : 1
```

```
plt.figure() # Create a new figure for the plot.
plt.bar(noun_freq.keys(), noun_freq.values()) # Generate a bar chart showing noun frequencies.
plt.title("Noun Frequency (Academic Concepts)") # Set the title of the plot.
plt.xlabel("Nouns") # Label the x-axis as 'Nouns'.
plt.ylabel("Frequency") # Label the y-axis as 'Frequency'.
plt.xticks(rotation=45) # Rotate x-axis labels for better readability.
plt.show() # Display the plot.
```




```
plt.figure() # Create a new figure for the plot.
plt.bar(verb_freq.keys(), verb_freq.values()) # Generate a bar chart showing verb frequencies.
plt.title("Verb Frequency (Arguments)") # Set the title of the plot.
plt.xlabel("Verbs") # Label the x-axis as 'Verbs'.
plt.ylabel("Frequency") # Label the y-axis as 'Frequency'.
plt.xticks(rotation=45) # Rotate x-axis labels for better readability.
plt.show() # Display the plot.
```


Verb Frequency (Arguments)
Frequency
1.0
0.8
0.6
0.4
0.2
0.0
enables
develop
support